Lab 2.b demo 3 walkthrough — the Claude hook (layer 2, authoring time)

The instructor's build-along demo ("Seven steps — build along on screen"), written down. Use it to replay the build at your own pace, to catch up if you missed the session, or to fix a step that didn't land live. Same rule as every demo walkthrough: this is the instructor's own script — there is no secret version.

You've seen this anatomy already. Demo 2's git hook was: an event fires a script, and the exit code decides. This demo moves that same anatomy to **authoring time** — the event is now the agent's tool call instead of your commit, the input is JSON on stdin instead of a message file, and the block is `exit 2` instead of any non-zero. Everything else is the shape you already know.

This covers **the demo guardrail only** (the read-only `db.json` rule). Your own clause — the one you pick from the menu — is the exercise; this document deliberately doesn't build it for you.

Before you start: your own copy, branch `feat/governance`, and the Claude Code **Hooks** lesson done.

Step 1 — Make the file exist

```
npm run reset-db
```

`db.json` is generated and git-ignored — a fresh copy doesn't have it, and a guardrail protecting a file that isn't there is hard to verify.

Step 2 — Layer 1: the instruction

One line in `CLAUDE.md`'s Don't section:

```
- Don't: edit `server/data/db.json` directly — it is generated; change data through
        the API, or edit seed.json via PR and run `npm run reset-db`
```

This informs. It does not enforce — that's the point of the next two layers.

Step 3 — Layer 2a: the deny rules

Merge into the existing `deny` array in `.claude/settings.json` (never paste over the file — your seed rules and hooks block must survive):

```
"deny": [
  "Edit(server/data/seed.json)",
  "Write(server/data/seed.json)",
  "Edit(server/data/db.json)",
  "Write(server/data/db.json)"
]
```

Step 4 — Layer 2b: the hook

Create `.claude/hooks/protect-db.js`. Same plumbing as `protect-seed.js` (read it first — the four-part message is the pattern):

```
#!/usr/bin/env node
let raw = '';
process.stdin.on('data', (c) => (raw += c));
process.stdin.on('end', () => {
  let filePath = '';
  try {
    filePath = JSON.parse(raw).tool_input?.file_path ?? '';
  } catch {
    process.exit(0); // unparseable — never block unrelated work
  }
  const path = filePath.replaceAll('\\', '/'); // Windows-safe

  if (path.endsWith('server/data/db.json')) {
    console.error(
      [
        'BLOCKED: db.json is generated, not edited.',
        'Why: hand-edits are lost on the next reset and are invisible to everyone
        else.',
        'Instead: change data through the API, or edit seed.json via PR and run npm
        run reset-db.',
        'Done means: your data change survives a reset.',
      ].join('\n'),
    );
    process.exit(2); // PreToolUse -> blocks
  }
  process.exit(0);
});
```

Note the message shape — **BLOCKED / Why / Instead / Done means**. A guardrail that only says “no” makes the agent guess, and it guesses creatively.

Step 5 — Register it, then start a NEW session

Add a second entry to the **existing** `PreToolUse` array (don't create a second `hooks` block):

```
"hooks": {
  "PreToolUse": [
    {
      "matcher": "Edit|Write",
      "hooks": [
        { "type": "command", "command": "node
        \"$CLAUDE_PROJECT_DIR/.claude/hooks/protect-seed.js\"" },
        { "type": "command", "command": "node
        \"$CLAUDE_PROJECT_DIR/.claude/hooks/protect-db.js\"" }
      ]
    }
  ]
}
```

Then **start a new Claude Code session** — hooks load at session start. "My hook never fires" is, nine times out of ten, this step skipped.

Step 6 — Verify the BLOCK (and hear the difference between your two layers)

In the new session, ask Claude to do the violation on purpose:

```
Open server/data/db.json and change one todo's title directly in the file.
```

First refusal — the wall. What answers is the **deny rule**, not your hook: a generic "denied by your permission settings" message. That's expected — permission rules evaluate *before* PreToolUse hooks get their turn (verified live). The agent knows it was stopped, but not why, and not what to do instead.

Second refusal — the signpost. Temporarily remove the two `db.json` deny entries from `.claude/settings.json` (leave the seed entries alone), start a **new session** (settings load at session start), and ask again. Now your hook answers:

```
BLOCKED: db.json is generated, not edited.
Why: hand-edits are lost on the next reset and are invisible to everyone else.
Instead: change data through the API, or edit seed.json via PR and run npm run reset-
db.
Done means: your data change survives a reset.
```

Save THIS refusal text — the hook's message is the evidence for your PR. Then **restore the two deny entries** (they'll load next session; your hook keeps protecting meanwhile). The difference you just

heard is the whole point: the deny is the cheap wall that stops the common case with no explanation; the hook is the signpost that redirects the agent — and only a hook can hold the *logic* your own clause needs this afternoon.

Finally, confirm the file is untouched — `db.json` is git-ignored, so `git status` won't mention it either way; just re-open the file and look (works the same on every OS).

Step 7 — Verify the ALLOW, then document the bypass

The half everyone skips:

```
Add a one-line comment to server/src/repositories/tags.repository.ts, then run npm
test -w server.
```

No refusal, no noise, suite green — the guardrail permits legitimate work. If this fails, the guardrail is worse than nothing: it now blocks work, and someone will remove it by Friday.

Finally, write the bypass down (in `CLAUDE.md`, next to the Don't line):

```
Bypass: seed/data changes go through a PR flagged to the instructor — never by
        disabling the hook.
```

The other verdict — what a nudge does to the agent

The hook you just built **blocks** (PreToolUse). The other severity, the **nudge** (PostToolUse), changes the workflow differently — and it's the shape most of the exercise menu uses:

1. The agent decides to edit a file and issues the tool call.
2. PreToolUse hooks get their veto. Assume allowed.
3. **The tool executes — the file is now changed on disk.** A PostToolUse hook can never undo that.
4. Your hook runs, reading the same JSON payload on stdin.
5. Exit 0: total silence — the agent never knows the hook exists. The 99% case.
6. Exit 2: the hook's **stderr is injected into the agent's conversation as feedback** — it reads the message mid-task, *before its next action*, and typically self-corrects on the spot: the review comment that used to cost a PR round-trip becomes an in-flight correction.

What the agent reads when a nudge fires (this is one of the exercise's menu rules speaking — the sibling-test nudge):

```
NOTE: server/src/services/stats.service.ts has no sibling test.
Why: CLAUDE.md's testing approach — tests are co-located; a change ships with its
test.
Instead: add or extend server/src/services/stats.service.test.ts before opening the
PR.
```

The agent's next move is usually to write that test — nobody asked, no review round-trip. **Building a hook like this is the exercise** — this section shows only what it feels like from the agent's side, and why a heuristic (PARTLY) rule should nudge: a false positive as a nudge costs one ignorable note; as a block it stops legitimate work and gets the hook disabled.

A testing trick that works for any hook — no live agent session needed. Hooks are just scripts reading JSON on stdin, so you can feed one a fake payload. Against the hook you just built:

macOS / Linux:

```
echo '{"tool_input":{"file_path":"server/data/db.json"}}' | node
        .claude/hooks/protect-db.js; echo "exit: $?"
```

Windows (PowerShell):

```
echo '{"tool_input":{"file_path":"server/data/db.json"}}' | node
        .claude/hooks/protect-db.js; echo "exit: $LASTEXITCODE"
```

You'll see the BLOCKED message and `exit: 2`. Swap the path for `server/src/repositories/tags.repository.ts` and you get silence and `exit: 0` — both verification directions, in two seconds, without restarting a session. (The live-agent verification in steps 6–7 is still the evidence standard — this trick is for fast iteration while you write the check.)

Common wobbles

Wobble	The move
Hook never fires	New session started? Hooks load at session start. Then: `matcher` says `Edit|Write`? Path check survives Windows backslashes?
Hook fires on the wrong files	Your path test is too broad — `endsWith('server/data/db.json')` not `includes('db.json')`, or you'll catch fixtures
The deny answers instead of my hook	Correct — permission rules evaluate before PreToolUse hooks. Step 6's two-message beat exists exactly for this: pull the deny, hear the hook, restore
Two `hooks` blocks in settings.json	Merge them — the file wants one `PreToolUse` array with two command entries
"It worked" with nothing to show	Not verified. The refusal text in your PR description is the standard